

Marco Thomas Marais

PROG7311

Student Number: ST10091902

Date: 27 March 2026

Word Count: ~1000

Introduction

TechMove Logistics currently relies on fragmented legacy systems consisting of spreadsheets, emails, and manual processes. This has resulted in data inconsistencies, lost invoices, compliance risks, and inefficient workflows. To address these challenges, the organisation aims to implement a Global Logistics Management System (GLMS)—an enterprise-grade, scalable, and distributed web platform.

The purpose of this report is to present a comprehensive architectural design for the GLMS before development begins. It applies enterprise architecture principles, identifies suitable design patterns, and evaluates critical non-functional requirements such as availability, interoperability, and scalability. The report ensures that the system can integrate with existing operations while supporting future growth and increased demand.

Architectural Framework Strategy (TOGAF)

To structure the GLMS, the TOGAF (The Open Group Architecture Framework) is selected. TOGAF provides a structured methodology through the Architecture Development Method (ADM), ensuring alignment between business objectives and IT systems.

Application of ADM Phases (A–D)

Phase A: Architecture Vision

This phase defines the high-level vision of the GLMS. The system will centralise logistics operations, enabling efficient contract management, service request processing, and financial integration. Stakeholders include logistics managers, clients, drivers, and financial administrators. The system must ensure high availability and scalability while eliminating inefficiencies caused by legacy systems.

Phase B: Business Architecture

The business architecture identifies the core processes of the GLMS:

- Contract management (creation, tracking, expiration monitoring)
- Service request processing linked to contracts
- Financial operations including invoicing and currency conversion
- Communication between stakeholders

This phase ensures that the system supports business workflows and eliminates bottlenecks present in the current system.

Phase C: Information Systems Architecture

Data Architecture

Key entities include:

- Clients
- Contracts
- Service Requests
- Invoices
- Drivers

Relationships between these entities ensure data consistency and traceability.

Application Architecture

The GLMS will adopt a microservices architecture, where components such as contract management, service processing, and financial integration operate as independent services. Communication will occur through REST APIs and message queues.

Phase D: Technology Architecture

The technology stack will include:

- Cloud infrastructure (AWS or Azure)
- Containerisation using Docker
- API gateways for service communication
- Load balancers for traffic distribution
- Caching systems (e.g., Redis)

This ensures high performance, scalability, and reliability.

Design Pattern Selection

To ensure maintainability and flexibility, the following **Gang of Four (GoF)** design patterns are implemented:

Model-View-Controller (MVC)

The MVC pattern separates the application into three components:

- **Model:** Handles data (Users, Products, Contracts)
- **View:** User interface (web/mobile UI)
- **Controller:** Business logic and request handling

This separation improves maintainability and allows independent development of components (Fowler, 2003).

Repository Pattern

The Repository Pattern abstracts data access logic and provides a consistent interface for interacting with data sources.

Application in GLMS:

- Manages CRUD operations for contracts, service requests, and invoices
- Separates business logic from database operations

This improves testability and flexibility (Evans, 2003).

Observer Pattern

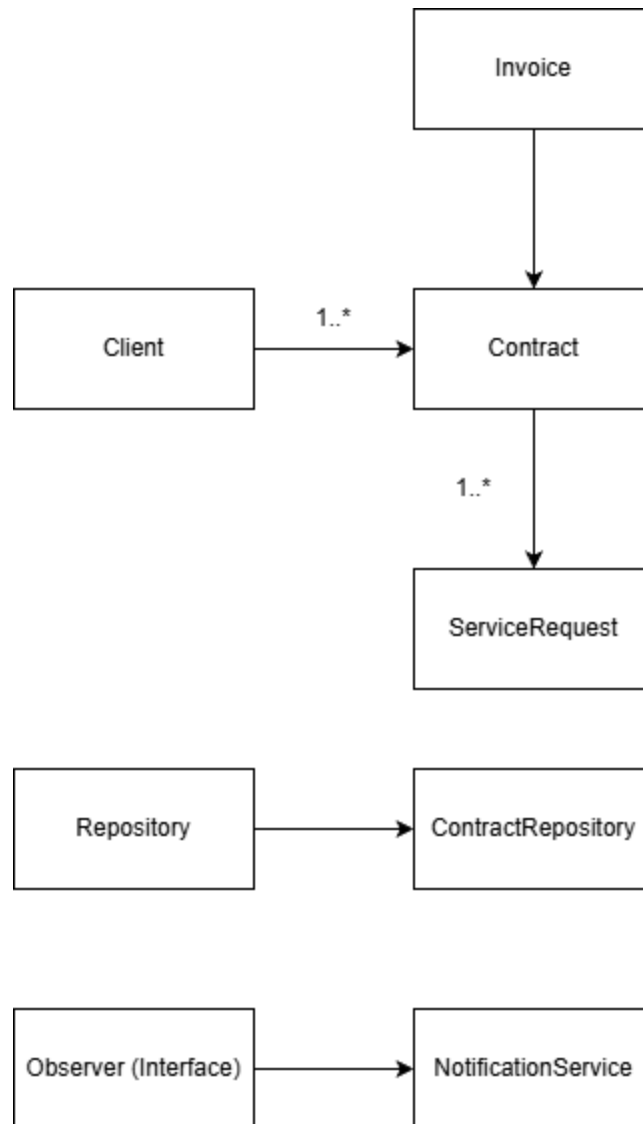
The Observer Pattern enables a one-to-many relationship where changes in one object trigger updates in others.

Application in project:

- Notification system for:
 - Contracts expire
 - Service requests are updated
 - Invoices are generated

This ensures real-time updates without continuous polling (Gamma et al., 1994).

UML Class Diagram (Conceptual Representation)



Non-Functional Requirements & Scalability

Availability

Availability refers to the system's ability to remain operational and accessible when required (Smith & Williams, 2002).

Importance:

The GLMS must be continuously available to support global logistics operations across different time zones. Downtime could result in delayed shipments and financial losses.

Implementation Strategy:

- Deploy across multiple cloud regions
- Use load balancers to distribute traffic
- Implement failover mechanisms

Impact on Development:

Developers must design fault-tolerant systems and ensure redundancy in infrastructure.

Interoperability

Interoperability is the ability of the system to integrate and communicate with external systems (Chen, 2008).

Importance:

The GLMS must integrate with:

- External financial APIs for currency conversion
- Third-party logistics systems
- Legacy data sources

Implementation Strategy:

- Use RESTful APIs and standard data formats (JSON/XML)
- Implement middleware for system integration
- Ensure API documentation and versioning

Impact on Development:

Developers must prioritise API design and ensure compatibility with external systems.

Scalability Strategy

Scalability refers to the system's ability to handle increased workloads without performance degradation (Chandra & Shenoy, 2001).

Scenario:

If TechMove acquires a competitor and adds 50,000 new contracts overnight, the system must handle the increased load efficiently.

Solution:

- **Horizontal Scaling:**

Additional servers can be deployed dynamically to handle increased demand.

- **Load Balancing:**

Incoming traffic is distributed across multiple servers to prevent overload.

- **Caching:**

Frequently accessed data (e.g., contract details) is stored in cache to reduce database load.

- **Database Scaling:**

Use sharding and replication to manage large datasets.

Impact on Development:

Developers must design stateless services and optimise database queries to support scaling.

Benefits of TOGAF for GLMS

The use of TOGAF provides significant advantages for the GLMS project. Firstly, it ensures alignment between business goals and technical implementation, reducing the risk of system failure. Secondly, the ADM cycle promotes a structured and repeatable development process, improving project governance and stakeholder communication. Additionally, TOGAF supports scalability by encouraging modular architecture design, which is essential for handling future growth such as increased contracts and users. Finally, it enhances risk management by identifying potential issues early in the architecture design phase, ensuring a more reliable and efficient system (The Open Group, 2018).

Conclusion

The Global Logistics Management System (GLMS) provides a modern solution to the inefficiencies faced by TechMove Logistics. By applying the TOGAF framework, the system is structured to align business goals with technical implementation. The use of design patterns such as MVC, Repository, and Observer ensures maintainability, flexibility, and scalability.

Additionally, the consideration of non-functional requirements such as availability, interoperability, and scalability ensures that the system can operate efficiently in a global environment while accommodating future growth. This architectural approach provides a strong foundation for developing a robust, enterprise-grade logistics platform.

References

Chandra, A. and Shenoy, P. (2001) 'Scalable Web Hosting Using Cooperative Caching and Load Balancing', *IEEE Transactions*.

Chen, D. (2008) 'Enterprise Interoperability Framework', *International Journal of Computer Integrated Manufacturing*.

Evans, E. (2003) *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley.

Fowler, M. (2003) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley.

Newman, S. (2015) *Building Microservices*. Sebastopol: O'Reilly Media.

Smith, C.U. and Williams, L.G. (2002) *Performance Solutions: A Practical Guide to Creating Responsive, Scalable Software*. Boston: Addison-Wesley.

The Open Group (2018) *TOGAF® Standard, Version 9.2*. Available at: <https://www.opengroup.org>